

A. TABLE CREATION

```
CREATE TABLE student (  
    student_id VARCHAR2(20) PRIMARY KEY,  
    student_name VARCHAR2(50),  
    course VARCHAR2(30)  
);
```

```
CREATE TABLE floor (  
    floor_id NUMBER PRIMARY KEY,  
    floor_name VARCHAR2(20),  
    total_seats NUMBER,  
    occupied_seats NUMBER DEFAULT 0,  
    priority_order NUMBER,  
    status VARCHAR2(10)  
);
```

```
CREATE TABLE seat (  
    seat_id NUMBER PRIMARY KEY,  
    floor_id NUMBER,  
    seat_no NUMBER,  
    status VARCHAR2(10),  
    FOREIGN KEY (floor_id) REFERENCES floor(floor_id)  
);
```

```
CREATE TABLE allocation (  
    allocation_id NUMBER PRIMARY KEY,  
    student_id VARCHAR2(20),
```

```
seat_id NUMBER,  
entry_time TIMESTAMP,  
exit_time TIMESTAMP,  
FOREIGN KEY (student_id) REFERENCES student(student_id),  
FOREIGN KEY (seat_id) REFERENCES seat(seat_id)  
);
```

```
CREATE TABLE scan_log (  
scan_id NUMBER PRIMARY KEY,  
student_id VARCHAR2(20),  
scan_type VARCHAR2(10),  
scan_time TIMESTAMP,  
floor_id NUMBER,  
FOREIGN KEY (student_id) REFERENCES student(student_id)  
);
```

B. SEQUENCES

```
CREATE SEQUENCE seat_seq START WITH 1;  
CREATE SEQUENCE alloc_seq START WITH 1;  
CREATE SEQUENCE scan_seq START WITH 1;
```

C. INSERT DATA

```
INSERT INTO floor VALUES (1,'Ground',200,0,1,'OPEN');  
INSERT INTO floor VALUES (2,'First',200,0,2,'OPEN');  
INSERT INTO floor VALUES (3,'Second',200,0,3,'OPEN');  
INSERT INTO floor VALUES (4,'Third',200,0,4,'OPEN');  
INSERT INTO floor VALUES (5,'Fourth',200,0,5,'OPEN');
```

```
INSERT INTO student VALUES ('STU101','Ayush','CSE');
```

```
INSERT INTO student VALUES ('STU102','Rahul','CSE');
```

```
COMMIT;
```

D. SEAT GENERATION

```
BEGIN
```

```
FOR f IN 1..5 LOOP
```

```
FOR i IN 1..200 LOOP
```

```
    INSERT INTO seat VALUES(seat_seq.NEXTVAL,f,i,'FREE');
```

```
END LOOP;
```

```
END LOOP;
```

```
END;
```

```
/
```

```
COMMIT;
```

E. ALLOCATION PROCEDURE

```
CREATE OR REPLACE PROCEDURE allocate_seat_floor(
```

```
    p_student_id IN VARCHAR2,
```

```
    p_floor_id  IN NUMBER
```

```
)
```

```
AS
```

```
    v_seat_id NUMBER;
```

```
    v_count  NUMBER;
```

BEGIN

-- Check student exists

SELECT COUNT(*) INTO v_count

FROM student

WHERE student_id = p_student_id;

IF v_count = 0 THEN

DBMS_OUTPUT.PUT_LINE('Student does not exist');

RETURN;

END IF;

-- Check if already inside

SELECT COUNT(*) INTO v_count

FROM allocation

WHERE student_id = p_student_id

AND exit_time IS NULL;

IF v_count > 0 THEN

DBMS_OUTPUT.PUT_LINE('Student already inside');

RETURN;

END IF;

-- Get free seat

SELECT seat_id INTO v_seat_id

FROM seat

WHERE floor_id = p_floor_id

AND status = 'FREE'

ORDER BY seat_no

```
FETCH FIRST 1 ROWS ONLY;
```

```
-- Insert allocation
```

```
INSERT INTO allocation
```

```
VALUES(alloc_seq.NEXTVAL, p_student_id, v_seat_id, SYSTIMESTAMP, NULL);
```

```
-- Update seat
```

```
UPDATE seat SET status = 'OCCUPIED'
```

```
WHERE seat_id = v_seat_id;
```

```
-- Update floor
```

```
UPDATE floor
```

```
SET occupied_seats = occupied_seats + 1
```

```
WHERE floor_id = p_floor_id;
```

```
DBMS_OUTPUT.PUT_LINE('Seat allocated to ' || p_student_id ||
```

```
    ' | Seat ID: ' || v_seat_id ||
```

```
    ' | Floor: ' || p_floor_id);
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('No seats available on selected floor');
```

```
END;
```

```
/
```

F. RELEASE PROCEDURE

```
CREATE OR REPLACE PROCEDURE release_seat(
```

```
p_student_id IN VARCHAR2
)
AS
    v_seat_id NUMBER;
    v_floor_id NUMBER;
BEGIN
    -- Get active allocation
    SELECT seat_id INTO v_seat_id
    FROM allocation
    WHERE student_id = p_student_id
    AND exit_time IS NULL;

    -- Mark exit
    UPDATE allocation
    SET exit_time = SYSTIMESTAMP
    WHERE student_id = p_student_id
    AND exit_time IS NULL;

    -- Get floor
    SELECT floor_id INTO v_floor_id
    FROM seat
    WHERE seat_id = v_seat_id;

    -- Free seat
    UPDATE seat SET status = 'FREE'
    WHERE seat_id = v_seat_id;

    -- Update floor
```

UPDATE floor

SET occupied_seats = occupied_seats - 1

WHERE floor_id = v_floor_id;

DBMS_OUTPUT.PUT_LINE('Seat released for ' || p_student_id ||

' | Seat ID: ' || v_seat_id ||

' | Floor: ' || v_floor_id);

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('No active allocation found for ' || p_student_id);

END;

/

G. TRIGGER

CREATE OR REPLACE TRIGGER scan_trigger

AFTER INSERT ON scan_log

FOR EACH ROW

BEGIN

IF UPPER(:NEW.scan_type) = 'ENTRY' THEN

allocate_seat_floor(:NEW.student_id, :NEW.floor_id);

ELSIF UPPER(:NEW.scan_type) = 'EXIT' THEN

release_seat(:NEW.student_id);

ELSE

DBMS_OUTPUT.PUT_LINE('Invalid scan type');

END IF;

END;

/